

Reviewer Pack — Minimal Geometric Entropy of Riemann Surfaces and Communicat...

Entry 1894e4bf1f8f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 19:31:40 UTC

Minimal Geometric Entropy of Riemann Surfaces and Communication Complexity of Polynomial Identities

Entry ID: 1894e4bf1f8f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 19:31:40 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry: Riemann Surfaces **Field B** (complexity object): Communication Complexity of Polynomial Identities

Statement:

{‘text’: ‘For every polynomial identity with variables $x_1, \dots, x_n$ over a field F of characteristic zero, the minimal geometric entropy of its solution set, defined as the minimum number of Riemann surfaces needed to cover it, is in polynomial time computable and upper bounds the deterministic communication complexity of deciding if a given point is a member of the solution set.’, ‘quantitative’: ‘ $E[H(\text{Solution_Set})] = \Theta(f(n))$ ’, ‘falsifier’: ‘For some polynomial identity with variables $x_1, \dots, x_n$ over $\bar{F}$, there exists a counterexample where the minimal geometric entropy of its solution set exceeds the deterministic communication complexity of deciding membership.’}

Rationale (proposer’s reasoning):

{‘text’: ‘Riemann surfaces provide a geometric representation of algebraic solutions that may reveal hidden structures in the communication process for polynomial identities. The minimal geometric entropy captures essential complexity features, potentially linking deep geometric properties to combinatorial communication complexity.’}

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2e0421f478490791

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal geometric entropy of the polynomial identity’s solution set, $E[H(\text{Solution_Set})]$, is computable in polynomial time and its value should not exceed the deterministic communication complexity for membership testing.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal geometric entropy" AND Riemann surfaces" - "communication complexity" AND polynomial identities" - "algebraic geometry" AND communication complexity"

Top relevant hits considered: - [http://arxiv.org/abs/2210.01601v2] Quantum communication complexity of linear regression - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/cs/9910010v2] Communication Complexity Lower Bounds by Polynomials - [http://arxiv.org/abs/1409.1534v1] Algorithms in Real Algebraic Geometry: A Survey - [http://arxiv.org/abs/1008.0601v3] A note on the cotangent complex in derived algebraic geometry - [http://arxiv.org/abs/math/0504259v1] Multiplier ideal sheaves in complex and algebraic geometry - [s2:10.1016/J.IJLEO.2021.166786] A lightweight encryption scheme using Chebyshev polynomial maps - [s2:10.1109/TIFS.2026.3657109] Efficient Revocable Conditional Anonymous Authentication With Verifiable Self-Generated Pseudonyms for VANETs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
import sys

def generate_polynomial(n):
    coefficients = [random.randint(-10, 10) for _ in range(n + 1)]
    return sum(coeff * x**i for i, coeff in enumerate(coefficients))

def evaluate_polynomial(poly, x):
    result = 0
    power_of_x = 1
    for coeff in poly:
        result += coeff * power_of_x
        power_of_x *= x
    return result

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.choice([5, 10, 15, 20, 30, 40])
```

```

polynomial = generate_polynomial(n)

# Simulate the solution set
solutions = []
for x in range(-100, 101):
    if evaluate_polynomial(polynomial, x) == 0:
        solutions.append(x)

# Minimal geometric entropy (number of Riemann surfaces needed to cover the solution set)
min_geometric_entropy = len(solutions)

# Simulate deterministic communication complexity for membership testing
def is_member(point):
    return evaluate_polynomial(polynomial, point) == 0

# Use a simple DPLL-like solver to simulate communication complexity
def dpll_solver():
    stack = []
    for x in range(-100, 101):
        if is_member(x):
            stack.append(x)
    return len(stack)

communication_complexity = dpll_solver()

# Check the conjecture
conjecture_holds = min_geometric_entropy <= communication_complexity

result = {
    "metric_name": "minimal_geometric_entropy",
    "metric_value": min_geometric_entropy,
    "instances_tested": len(solutions),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Counterexample found: n={n}, polynomial={p}"
}

return result

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(res["metric_value"] for res in results) / len(results)
    std_metric_value = math.sqrt(sum((res["metric_value"] - mean_metric_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif sum(1 for res in results if not res["conjecture_holds"]) / len(results) >= 0.2:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='Counterexample found' first_failing_seed={first_failing_seed}")

```

```

else:
    print("RESULT: INCONCLUSIVE insufficient evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c6b40ce2.py", line 77, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c6b40ce2.py", line 34, in run_trial
    polynomial = generate_polynomial(n)
                  ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c6b40ce2.py", line 20, in generate_poly
    return sum(coeff * x**i for i, coeff in enumerate(coefficients))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c6b40ce2.py", line 20, in <genexpr>
    return sum(coeff * x**i for i, coeff in enumerate(coefficients))
                  ^
NameError: name 'x' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14123
2	preregistration	ollama_remote	glm4:latest	0	0	9311
3	novelty	ollama_remote	glm4:latest	0	0	8668
4	novelty	ollama_remote	glm4:latest	0	0	9879
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12311
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8872
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8380
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9015

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	12720

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 93278 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/1894e4bf1f8f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/1894e4bf1f8f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1894e4bf1f8f.tar.gz (if generated)